

CS205 C/ C++ Programming - Project 2 Simple Matrix Multiplication

Name: 殷海洲(Yin Haizhou)

SID: 12210226

Part 1 - Analysis

Requirement

1. Implement a program in C and another in Java for matrix multiplication. You can implement a function for the multiplication. Only `float` is needed since it is the most commonly used data type in deep learning.
2. Measure the time of the computation. As the sizes of the matrices increase, the time should increase. Measuring the time is not easy since many factors will affect it.
3. Compare the performance of the two programs in C and Java, and explain the reason for any observed differences.
4. Perform additional comparisons and analysis to identify any interesting insights.

Initially, we need to understand the basic knowledge about matrices and their multiplication rules which indicates latent constraints for two matrices is that the number of columns of the first matrix equals the number of rows of the second one. Before computation, it is important to make sure that the matrices are valid.

Then, we need a function to generate matrices of random numbers and implement computation.

Last, we measure the time it costs to multiply two matrices and compare the time cost in C and Java.

The main structure is described as above. To be more detailed, I would like to discuss how it could be realized.

Structure Configuration

How I design my program

Users input desired numbers of columns and rows of two matrices, and the program measures its validity as soon as it operates. It terminates immediately if it is invalid to avoid more problems.

A new structure(in C or class in Java) for matrices will be created and include three members which are their numbers of rows and columns, and `pdata`, which are pointers for address of dynamic storage. Here, it is emphasized with significance of using pointers such that we could allocate adequate memories for uncertain data.

A structure array is created including matrices being computed and a result matrix. It would help in latter operationa when a iterative operation is required so we can avoid redundant repeat of codes but a clean loop instead.

A random matrix is generated by first grabbing time variable as a random seed using `time.h`, which offers many tools required. It generates a number in each loop, maps every single space and fills with random data.

The data of matrices are special since they are not nested arrays but one-dimension lists in storage. So we also use pointer to map each number for calculation. If we know the address `address` of a certain matrix, the number of (m,n) will be referred as `address+(n-1)*m+m-1`, but be cautious the pointers need to be `float *`.

Time cost will be counted only for the calculation.

It is crucial to be aware of freeing memories allocated. Hence it would be better to understand explicitly which to free and if I have freed.

Part 2 - Code

This part is for code and I would like to mention that my source code is based on three files including one main function and two more functions concerning manipulations on matrices. They are originally compiled and linked using makefile.

Meanwhile, the .c file I submitted is a integrated version, which is almost the same as the previous version. Therefore, if you see some differences between the code in report and the code in my file, please ignore that.

C version

According to our analysis, now we need to first figure out an appropriate structure to include information of matrices.

```
typedef struct _Matrix{  
    int row;  
    int col;  
    float * pdata;  
}Matrix;
```

A matrix structure is of three parameters, which are column and row numbers, and a pointer refers to its data address instead of itself.

We also used `typedef` to avoid repeatedly using `struct` .

Then, we using multiple loops and random function to create a matrix.

Matrix generator

```
void geneM(Matrix *ptr)
{
    srand(time(NULL));
    for(int i = 0; i < (ptr->row); i++)
    {
        for(int j = 0; j < (ptr->col); j++)
        {
            float rand_float = ((float)rand() / RAND_MAX);
            ptr->pdata[j + i*(ptr->col)] = rand_float;
        }
    }
}
```

In loops, all positions for float number will be mapped and be assigned with a random number.

The float number generated will be in range of (0,RAND_MAX).

Then, we define a function to multiply two matrices. Here, we using pointers pointing at the matrices themselves to simulate [reference](#) which is not equipped in C language.

The benefit of using pointers will be that it will transfer only address instead of every single elements inside matrices to reduce waste in storage.

Multiplication function

```

void mult(const Matrix *ptrA, const Matrix *ptrB, Matrix *ptrC)
{
    clock_t start,end;
    double time;
    if ((*ptrA).col != (*ptrB).row || ptrA == NULL || ptrB == NULL || ptrC == NULL)
        printf("Matrices are invalid! Be careful about that A column equals B row.");
    else{
        start = clock();
        for(int i = 0; i < (*ptrB).col; i++)//varying B column
        {
            for(int j = 0; j < (*ptrA).row; j++)//varying A row
            {
                for (int k = 0; k < (*ptrA).col; k++)//varying A column and B row
                {
                    (*ptrC).pdata[i+j*(*)ptrC).col] = 0;
                    //C[j][i]
                    (*ptrC).pdata[i+j*(*)ptrB).col] += (*ptrA).pdata[(*)ptrA).col*j+k] * (*ptrB).p
                }
            }
        }
        end = clock();
        time = ((double) (end - start)) / CLOCKS_PER_SEC;
        printf("It costs %lf second.\n",time);
    }
}

```

Also, we count the time cost in the process of multiplying.

Eventually, we combine and apply those functions into a complete source code.

```

int main()
{

    int row1, row2, col1, col2;
    printf("Please input a number as the row of your first matrix\n");
    scanf("%d",&row1);
    printf("Please input a number as the column of your first matrix\n");
    scanf("%d",&col1);

```

```

printf("Please input a number as the row of your second matrix\n");
scanf("%d",&row2);
printf("Please input a number as the row of your second matrix\n");
scanf("%d",&col2);

int size[6] = {row1,row2,col1,col2,row1,col2};

if (col1 != row2 || row1 <= 0 || row2 <= 0 || col1 <= 0 || col2 <= 0)
{
    printf("Invalid matrix. Please check your columns and rows.\n");
    return 1;
}
else
{
    Matrix matrix[3];
    Matrix *ptr1 = &matrix[0];
    Matrix *ptr2 = &matrix[1];
    Matrix *ptr3 = &matrix[2];
    Matrix *ptr[3] = {ptr1,ptr2,ptr3};

    for(int i = 0;i < 3;i++)
    {
        (*ptr[i]).row = size[2*i];
        (*ptr[i]).col = size[2*i+1];
        (*ptr[i]).pdata = (float*)malloc((*ptr[i]).row * (*ptr[i]).col*sizeof(float));
        if ((*ptr[i]).pdata == NULL)
            printf("Wrong allocation; Fault Gate 1.");
    }

    geneM(ptr1);
    geneM(ptr2);
    mult(ptr1,ptr2,ptr3);

    for(int i = 0;i < 3;i++)
    {
        if ((*ptr[i]).pdata != NULL)
        {
            free((*ptr[i]).pdata);
            printf("Matrix info freed.\n");
            (*ptr[i]).pdata = NULL;
        }
    }
}

```

```
    }  
}  
return 0;  
}
```

Generally speaking, we first request the user's input on matrix size and we allocate memory for matrices. Then, we generate data inside matrices and calculate them while counting time costs. Finally, we free the memory allocated and the program is over.

Preprocess:

```
#include<stdio.h>  
#include<stdlib.h>  
#include<float.h>  
#include <time.h>
```

Java version

The code structure of java version is similar to that of c version in how we process them.

To have a better comparison with C version, we tried to make them as the same as possible so that their difference will mostly be how they calculate matrices and how fast they are originally supposed to be.

```
import java.util.Random;

class Matrix {
    int row;
    int col;
    float[] pdata;
}

public class Main {
    public static void main(String[] args) {
        int row1, row2, col1, col2;
        java.util.Scanner scanner = new java.util.Scanner(System.in);
        System.out.println("Please input a number as the row of your first matrix");
        row1 = scanner.nextInt();
        System.out.println("Please input a number as the column of your first matrix");
        col1 = scanner.nextInt();
        System.out.println("Please input a number as the row of your second matrix");
        row2 = scanner.nextInt();
        System.out.println("Please input a number as the column of your second matrix");
        col2 = scanner.nextInt();

        int[] size = {row1, row2, col1, col2, row1, col2};

        if (col1 != row2 || row1 <= 0 || row2 <= 0 || col1 <= 0 || col2 <= 0) {
            System.out.println("Invalid matrix. Please check your columns and rows.");
            return;
        } else {
            Matrix[] matrix = new Matrix[3];
            Matrix matrix1 = new Matrix();
            Matrix matrix2 = new Matrix();
            Matrix matrix3 = new Matrix();
            matrix[0] = matrix1;
            matrix[1] = matrix2;
            matrix[2] = matrix3;

            for (int i = 0; i < 3; i++) {
                matrix[i].row = size[2 * i];
                matrix[i].col = size[2 * i + 1];
                matrix[i].pdata = new float[matrix[i].row * matrix[i].col];
                if (matrix[i].pdata == null)
```



```

        System.out.println("Wrong allocation; Fault Gate 1.");
    }

    geneM(matrix1);
    geneM(matrix2);
    mult(matrix1, matrix2, matrix3);
}

static void geneM(Matrix ptr) {
    Random rand = new Random();
    for (int i = 0; i < ptr.row; i++) {
        for (int j = 0; j < ptr.col; j++) {
            float rand_float = rand.nextFloat();
            ptr.pdata[j + i * ptr.col] = rand_float;
        }
    }
}

static void mult(Matrix A, Matrix B, Matrix C) {
    long start, end;
    double time;
    if (A.col != B.row)
        System.out.println("Matrices are invalid! Be careful about that A column equals B row");
    else {
        start = System.nanoTime();
        for (int i = 0; i < B.col; i++) {
            for (int j = 0; j < A.row; j++) {
                for (int k = 0; k < A.col; k++) {
                    C.pdata[i + j * C.col] = 0;
                    C.pdata[i + j * B.col] += A.pdata[A.col * j + k] * B.pdata[B.col * k + i];
                }
            }
        }
        end = System.nanoTime();
        time = (double) (end - start) / 1000000000.0;
        System.out.println("It costs " + time + " second.");
    }
}
}

```

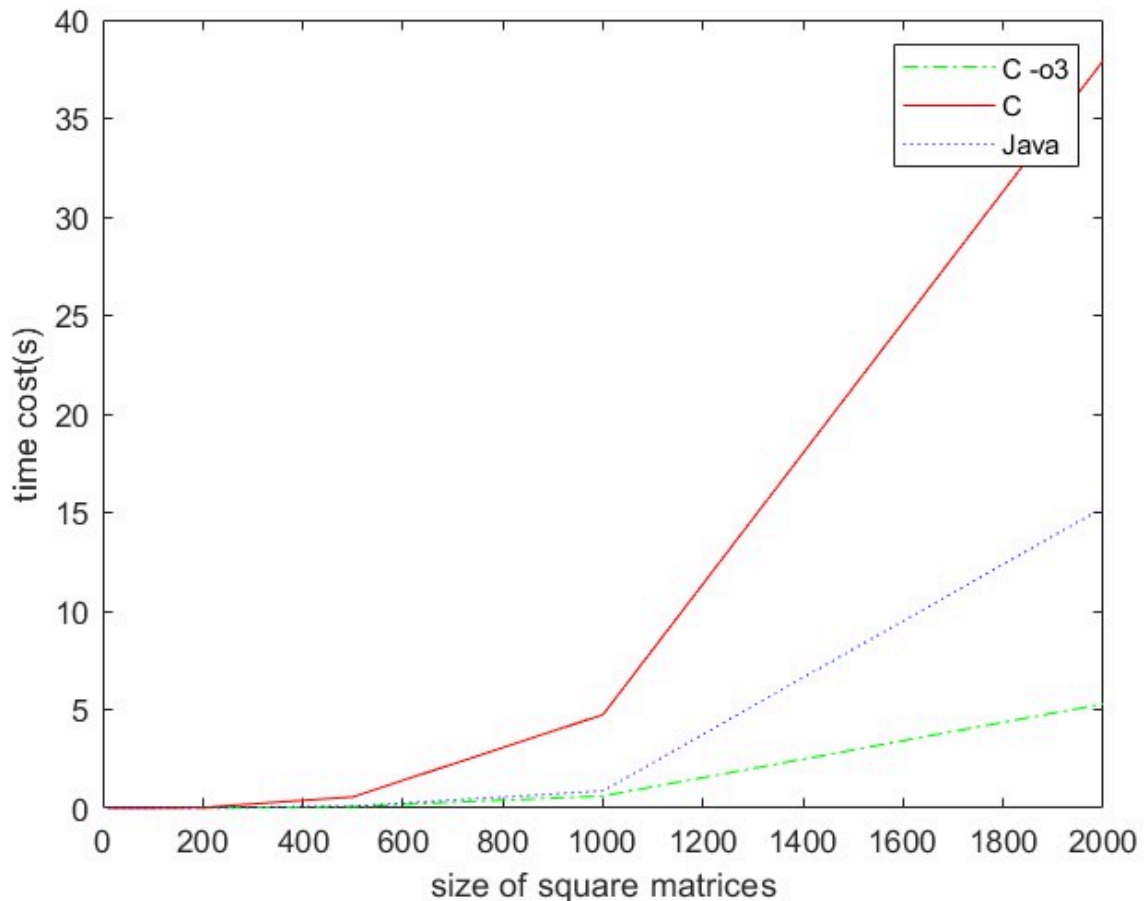
Part 3 - Result & Verification

\$1. Comparisons between C and Java programs

We assume all matrices are squares, and then we implement different lateral length from {10,20,50,100,200,500,1000,2000} into a C program with Optimization 3, a C program and a Java program. We try multiple times and get statistics and draw a figure.

	Co3_AVERAGE	C_AVERAGE	Java_AVERAGE
10	0.0000014	0.0000064	0.0000315
20	0.0000048	0.0000438	0.0002402
50	0.0000836	0.0007122	0.0024
100	0.000632	0.0046884	0.005
200	0.0042788	0.036991	0.0128
500	0.081253	0.5708518	0.1044
1000	0.6167598	4.7451814	0.8742
2000	5.291987	37.939557	15.249

(unit: second)



Analysis

C program with o3 optimization is the fastest, and Java program is less fast while C program is least fast. This is because probably compiler of C program reduced some redundant operations in programs while reducing the burden caused by functions or even expanded some loops to accelerate. In contrast, Java program is based on a fairly different environment compared with C language, which enables Java codes to be simplified and run faster. At last, C program is simply compiled with little optimization, and that will result in some inessential operation and extend running time.

\$2. Factors influencing time cost

Square demonstrates speed between different language, and it is not complete because matrices are not all squares. Therefore, we now implement matrices of the same data size to observe their difference in multiplication.

```
a = [0.835949 0.117903 0.007752 0.003459 0.000531 0.000089 0.000041 0.000008 0.000005];  
%% 10000x1 5000x2 1000x10 500x20 100x100 20x500 10x1000 2x5000 1x10000
```

We surprisedly found that even if matrices are of the same data size(meaning that they have same number of numbers in each matrix), it is evident that the first matrix of more row number ran much slower than those of less row number.

This is because fundamentally, when matrix A($m \times n$) multiply with matrix B($n \times m$), there will undergo operations for a total number of $(2n-1) \times m^2$ times. When $m \times n$ is a constant number, then it becomes $(2\text{Const}-m) \times m$, which implies only m influence time of calculation. Consequently, if $m \rightarrow$ too large, the multiplying time costs increase.

Besides, we made three assumptions as additional possible reasons for such a phenomenon.

1. The speed was highly influenced by loops.
2. The speed was highly influenced by writing.
3. The speed was highly influenced by reading.

@1 To prove that the first assumption is correct, we use a simple code like

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>

int main()
{
    clock_t start,end1,end2,end3;
    double time1,time2,time3;
    int b = 0;
    int c = 0;
    int d = 0;
    start = clock();
    for (int i=0;i<1000;i++)
        b = b + 10 + 10 + 10 + 10 + 10 + 10 + 10 + 10 + 10 + 10;
    end1 = clock();
    for (int i = 0;i<1000;i++)
    {
        for (int j = 0;j<10;j++)
            c += 10;
    }
    end2 = clock();
    for (int i = 0;i<10;i++)
    {
        for (int j = 0;j<1000;j++)
            d += 10;
    }
    end3 = clock();
    time1 = ((double) (end1 - start)) / CLOCKS_PER_SEC;
    time2 = ((double) (end2 - end1)) / CLOCKS_PER_SEC;
    time3 = ((double) (end3 - end2)) / CLOCKS_PER_SEC;
    printf("It costs %lf second.\n",time1);
    printf("It costs %lf second.\n",time2);
    printf("It costs %lf second.\n",time3);
}
```

From the result, we see more loops result in more time cost.

```

● chlorine@DESKTOP-CR08L6M:/mnt/e/C++/project2$ ./a.out
It costs 0.000002 second.
It costs 0.000018 second.
It costs 0.000020 second.
● chlorine@DESKTOP-CR08L6M:/mnt/e/C++/project2$ ./a.out
It costs 0.000002 second.
It costs 0.000019 second.
It costs 0.000019 second.

```

Enlarge ceils of i and j to ten times, and we will get:

```

● chlorine@DESKTOP-CR08L6M:/mnt/e/C++/project2$ ./a.out
It costs 0.000191 second.
It costs 0.001800 second.
It costs 0.001894 second.

```

Without optimization, it is explicit that with the number of loops increasing, the time cost increase. However, there is only a slight difference between time of the same total repeated times but different repeated times for each loop. Namely, there is a tendency that more repeats inside loops, it costs negligibly more.

Therefore, loops may not influence much to the time cost.

@2. To prove the second assumption is correct, we need

```

#include <stdio.h>
#include <time.h>

int main()
{
    double time1, time2;
    clock_t start, end1, end2;
    start = clock();
    int a = 0;
    int *p = &a;
    for (int i = 0; i < 1000000; i++)
        *p = i;
    end1 = clock();
    for (int i = 0; i < 10; i++)
    {
        for (int j = 0; j < 100000; j++)

```

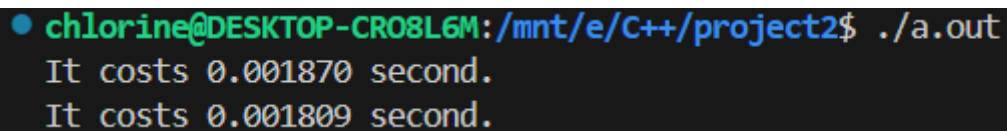
```

        *p = j;
    }
    end2 = clock();
    time1 = ((double) (end1 - start)) / CLOCKS_PER_SEC;
    time2 = ((double) (end2 - end1)) / CLOCKS_PER_SEC;
    printf("It costs %lf second.\n",time1);
    printf("It costs %lf second.\n",time2);
}

```

This part indicates time spent on writing doesn't vary according to the shape of matrices.

We get the result



```

chlorine@DESKTOP-CR08L6M:/mnt/e/C++/project2$ ./a.out
It costs 0.001870 second.
It costs 0.001809 second.

```

The result indicates that writing does not influence speed a lot.

@3 To prove that the third assumption is correct, we first elaborate why third assumption.

I made a guess that programs working resembles people using automatic rifles. That is, we load bullets(data) first and then we shoot(run program). Since it is inefficient to load one bullet and shoot every time, we may using something like clips to load a lot of bullets in one go. I believe it might be the same for computer to load many data in a cache to accelerate the program.

Thus, in my source code of generating numbers of matrices, they are arranged linearly in memory and follows an order of row-priority(left to right, up to down). Thus, when I use them, computer preferentially load row data(left to right), it will be fast to undergo calculation in this order. Nevertheless, if calculation is accomplished from up to down, it might probably be slower.

To prove it does matter, we change the way that we used to arrange matrices numbers from row priority to column priority, and again compile and link with other files.

```

void geneM(Matrix *ptr)
{
    srand(time(NULL));
    for(int j = 0; j < (ptr->col); j++)
    {
        for(int i = 0; i < (ptr->row); i++)
        {
            float rand_float = ((float)rand() / RAND_MAX);
            ptr->pdata[j + i*(ptr->col)] = rand_float;
        }
    }
}

```

This code uses a column-prior method to arrange numbers in matrices.

```

● chlorine@DESKTOP-CR08L6M:/mnt/e/C++/project2$ ./multiply
Please input a number as the row of your first matrix
10000
Please input a number as the column of your first matrix
10
Please input a number as the row of your second matrix
10
Please input a number as the row of your second matrix
10000
It costs 1.185681 second.
Matrix info freed.
Matrix info freed.
Matrix info freed.
● chlorine@DESKTOP-CR08L6M:/mnt/e/C++/project2$ ./multiply2
Please input a number as the row of your first matrix
10000
Please input a number as the column of your first matrix
10
Please input a number as the row of your second matrix
10
Please input a number as the row of your second matrix
10000
It costs 1.113938 second.
Matrix info freed.
Matrix info freed.
Matrix info freed.

```

We could find it has become apparently faster.

Going through the Internet, we found this is resulting from concepts of Sequential Access and Random Access, and they influence Cache and Prefetching, which further influence speed and time cost. Furthermore, Java programs also seem to be influenced, while some computer languages based on special environments are not.

Part 4 - Difficulty & Solution

1. The very first thing of the whole project is to consider how to multiply two matrix. At first I tried to implement a 2D array for each matrix which is straightforward and simple to understand. However, it is not simple at all to manipulate the 2D array and generate arbitrary size of 2D array. Hopefully, it reminded me of address manipulation and dynamic memory allocation so I chose using pointers to replace an array.
2. Pointer manipulation always brings about bugs, and particularly, segmentation fault. To avoid those bugs it is regarded to be important that we should carefully check if pointers are valid for the following operation, as well as what value we are returning by pointers, and finally, if it is about to cause memory leak. Hence, I frequently checked status of pointers in my program.
3. There are many factors influence the time cost. In order to figure them out, we apply both mathematic analysis and abstract analysis. Finally, we found as for matrices multiplication, the size of matrices mainly influences time cost while the arrangement of numbers in matrices also can be an inferior factor of time cost.

Appendix

experience, comments, and reference

Experience and Comments

I used pointer arrays to avoid ugly coding style, though I think it might be dangerous.

I was keeping seeking something interesting, but I didn't even understand something more crucial than coding for matrix multiplication so that's terrible.

It was the best that I had had only ten bugs for my first version, which is incredible.

I decided to use makefile to compile and link while our teacher appears to ask us to upload only one *.c file.

I hope the projects in the future may explicitly indicates their specific requirements.

Java is really similar to C/C++, except for `new`.

I coded something that can show what is inside the result matrix. Be careful unless you want to see millions of numbers printed in the terminal.

(If to check the result)

```
for (int i = 0; i < (*ptr3).row; i++)
{
    for (int j = 0; j < (*ptr3).col; j++)
        printf("%f ", (*ptr3).pdata[(*ptr3).row*i+j]);
    printf("\n");
}
```

Reference

<https://github.com/ShiqiYu/Cpp>

<https://github.com/YeeTone/CS205-2020Fall>

<https://zhuanlan.zhihu.com/p/86513504>

ChatGPT 3.5